# PropeMange (Property App) - Technical Documentation

## 1. Overview

**PropeMange** is a modern, cross-platform real estate and property management application built using Flutter. The app allows users to authenticate, list their properties, and browse available properties. It features a map-based search, voice recognition for search capabilities, and a detailed property browsing experience.

## 2. Technology Stack

The application leverages a robust set of modern Flutter packages and services:

- **Framework:** Flutter (SDK `^3.10.7`)
- **Backend:** Firebase (Authentication, Cloud Firestore, Firebase Storage)
- **State Management:** Provider
- **Dependency Injection:** GetIt
- **Data Equality:** Equatable
- **Maps & Location:** Google Maps Flutter, Geolocator
- **Voice Recognition:** Speech To Text
- **UI Utilities:** Cached Network Image (for efficient image loading), Intl (for date/number formatting), Cupertino Icons.

## 3. Architecture

The project strictly adheres to **Clean Architecture** principles, organizing code into isolated layers to promote separation of concerns, testability, and scalability. The application is feature-driven and divided into `core` and `features` directories.

### Project Structure

```
lib/
├── core/                  # Contains shared utilities, exceptions, network clients, etc.
├── features/
│   ├── auth/              # Authentication Feature
│   │   └── presentation/  # UI components like signin_view.dart
│   └── property/          # Property Management Feature
│       ├── data/          # Data Layer (Models, Repositories, Data Sources)
│       ├── domain/        # Domain Layer (Entities, Use Cases, Repository Interfaces)
│       └── presentation/  # Presentation Layer (Pages, Widgets, Providers)
└── main.dart              # Entry point of the application
```

### Layer Details

1. **Domain Layer (Core Business Logic):**
    - **Entities:** Represents the core data structures (e.g., `PropertyEntity` defining `id`, `ownerId`, `title`, `price`, `locationAddress`, `latitude`, `longitude`, etc.).
    - **Use Cases:** Contains application-specific business rules.
    - **Repositories (Interfaces):** Abstract contracts defining how data is accessed.
2. **Data Layer (Data handling):**
    - **Models:** Data structures extending Entities with serialization/deserialization capabilities (e.g., `PropertyModel` which handles JSON/Firestore conversion).
    - **Repositories (Implementations):** Concrete implementations of domain repository interfaces, interacting with data sources (Firebase Firestore, Storage).
3. **Presentation Layer (UI and State):**
    - **Pages:** The main screens of the application (`landing_view.dart`, `home_view.dart`, `property_details_view.dart`, `search_view.dart`, `add_property_view.dart`, `account_view.dart`).
    - **Widgets:** Reusable UI components used across the pages.
    - **Providers:** State management classes (`PropertyProvider`) connecting the UI to the domain layer.

## 4. Key Features & Screens

### Authentication

- **Sign In View (`signin_view.dart`):** Handles user login, typically interacting with Firebase Authentication.

### Property Management

- **Landing View (`landing_view.dart`):** The initial screen presented to the user, acting as the root navigation or welcome screen.
- **Home View (`home_view.dart`):** The main dashboard displaying recommended or recently listed properties.
- **Search View (`search_view.dart`):** Allows users to search for properties. Integrated with `google_maps_flutter` for map-based discovery and `speech_to_text` for voice searching.
- **Property Details View (`property_details_view.dart`):** Displays comprehensive information about a selected property, including an image gallery (using `cached_network_image`), pricing, location, and specifications (beds, baths, sqft).
- **Add Property View (`add_property_view.dart`):** A form screen enabling authenticated users to list their own properties. It likely uses Firebase Storage for uploading property images and Firestore for saving the listing data.
- **Account View (`account_view.dart`):** User profile management, displaying user details and potentially their own listed properties or saved favorites.

## 5. Data Models

### Property Entity

The core representation of a property in the application is defined as `PropertyEntity`:

```dart
class PropertyEntity extends Equatable {
  final String id;
  final String ownerId;
  final String title;
  final String description;
  final double price;
  final String category;
  final String locationAddress;
  final List<String> images;
  final int beds;
  final int baths;
  final double sqft;
  final double latitude;
  final double longitude;
  // ...
}
```

## 6. Setup and Initialization

To run this application locally:

1. Ensure the Flutter SDK `^3.10.7` is installed.
2. The app depends on Firebase. The `main.dart` is structured to initialize Firebase (`Firebase.initializeApp()`), which requires appropriate `google-services.json` (Android) and `GoogleService-Info.plist` (iOS) files to be configured.
3. Google Maps requires API keys to be configured in both the Android `AndroidManifest.xml` and iOS `AppDelegate.swift`.
4. Run `flutter pub get` to fetch all dependencies.
5. Execute the app using `flutter run`.